

Definition of JOIN in MySQL

A **JOIN** in MySQL is used to combine data from **two or more tables** based on a **related column** between them (usually a **foreign key**).

It allows you to retrieve data that exists **across multiple tables** as if they were one.

Sample Database and Tables

Let's create a small database and two related tables: students and courses.

Create Database

```
CREATE DATABASE school_db;
```

```
USE school_db;
```

Create Tables

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    student_name VARCHAR(50),  
    course_id INT  
);
```

```
CREATE TABLE courses (  
    course_id INT PRIMARY KEY,  
    course_name VARCHAR(50)  
);
```

Insert Sample Data

```
INSERT INTO students VALUES  
(1, 'John', 101),  
(2, 'Emma', 102),  
(3, 'Ali', 103),  
(4, 'Sara', NULL);
```

```
INSERT INTO courses VALUES
(101, 'MySQL Basics'),
(102, 'Python Programming'),
(103, 'Data Analytics'),
(104, 'Excel Advanced');
```

Types of Joins in MySQL

INNER JOIN

Returns **only matching rows** from both tables.
(i.e., students who have a course assigned that exists in the course table)

Syntax:

```
SELECT students.student_name, courses.course_name
FROM students
INNER JOIN courses
ON students.course_id = courses.course_id;
```

Output:

student_name	course_name
John	MySQL Basics
Emma	Python Programming
Ali	Data Analytics

Sara is **not shown** because her course_id is NULL.

LEFT JOIN (or LEFT OUTER JOIN)

Returns **all rows from the left table** (students) and **matching rows** from the right (courses).
If no match, NULL is shown for the right table.

Syntax:

```
SELECT students.student_name, courses.course_name
FROM students
```

LEFT JOIN courses

ON students.course_id = courses.course_id;

Output:

student_name	course_name
John	MySQL Basics
Emma	Python Programming
Ali	Data Analytics
Sara	NULL

Sara appears, but with NULL course.

RIGHT JOIN (or RIGHT OUTER JOIN)

Returns **all rows from the right table** (courses) and **matching rows** from the left (students).

Syntax:

SELECT students.student_name, courses.course_name

FROM students

RIGHT JOIN courses

ON students.course_id = courses.course_id;

Output:

student_name	course_name
John	MySQL Basics
Emma	Python Programming
Ali	Data Analytics
NULL	Excel Advanced

Here, **Excel Advanced** has **no student enrolled**.

FULL JOIN (Full Outer Join)

Returns **all rows from both tables**, with NULL where there's no match.

MySQL **does not support FULL JOIN** directly.

You can simulate it using **UNION** of LEFT and RIGHT joins:

```
SELECT students.student_name, courses.course_name
FROM students
LEFT JOIN courses
ON students.course_id = courses.course_id
```

UNION

```
SELECT students.student_name, courses.course_name
FROM students
RIGHT JOIN courses
ON students.course_id = courses.course_id;
```

CROSS JOIN

Returns **all possible combinations** (Cartesian product) of both tables.

Syntax:

```
SELECT students.student_name, courses.course_name
FROM students
CROSS JOIN courses;
```

Output: 4 students × 4 courses = 16 combinations.

Summary Table

Join Type	Description	Returns
INNER JOIN	Only matching rows	Matches from both tables
LEFT JOIN	All from left, matched from right	NULL if no match
RIGHT JOIN	All from right, matched from left	NULL if no match
FULL JOIN	All from both	NULLs where no match
CROSS JOIN	All combinations	Cartesian product